Project information

Jens Lahti, Student ID = 0176

Target grade = 3

https://github.com/jenslahti/portfolio-website

Features

- Feedback form including five input fields: Full name, email, topic, message and website.

- Field validation showing clear error messages.

- Required and optional fields, including specific input requirements.

- Form showing visual states for neutral, error or valid input.

- Live character counter for message input.

- Success message for user. Clearing after 4 seconds per assignment requirement.

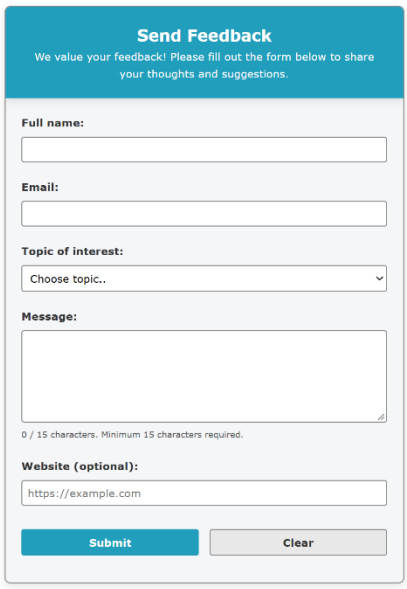
- Submit and Clear button.

- No data is sent or stored.

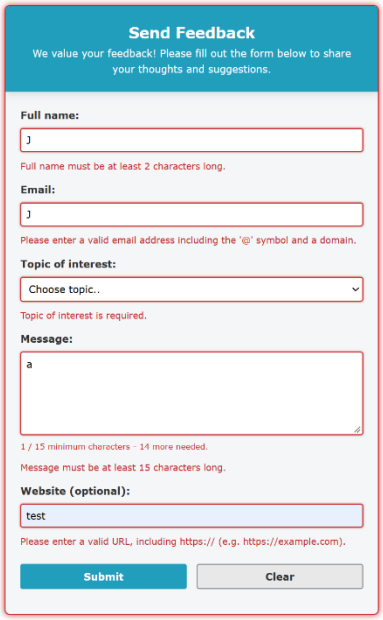
Feature evidence

Neutral state, no red/green colors, no error/success messages

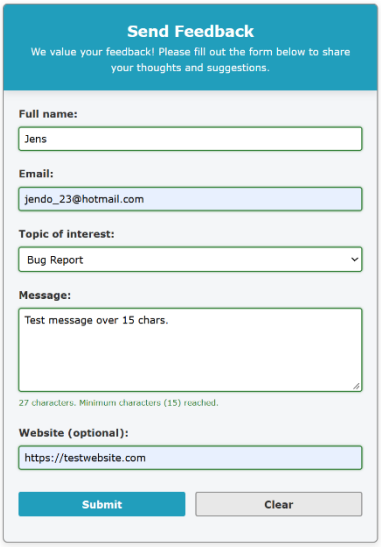
---



Error state: submitted with name “J”, email “J”, no topic selected and a 1-character message. Each field shows a red border and a specific error message.

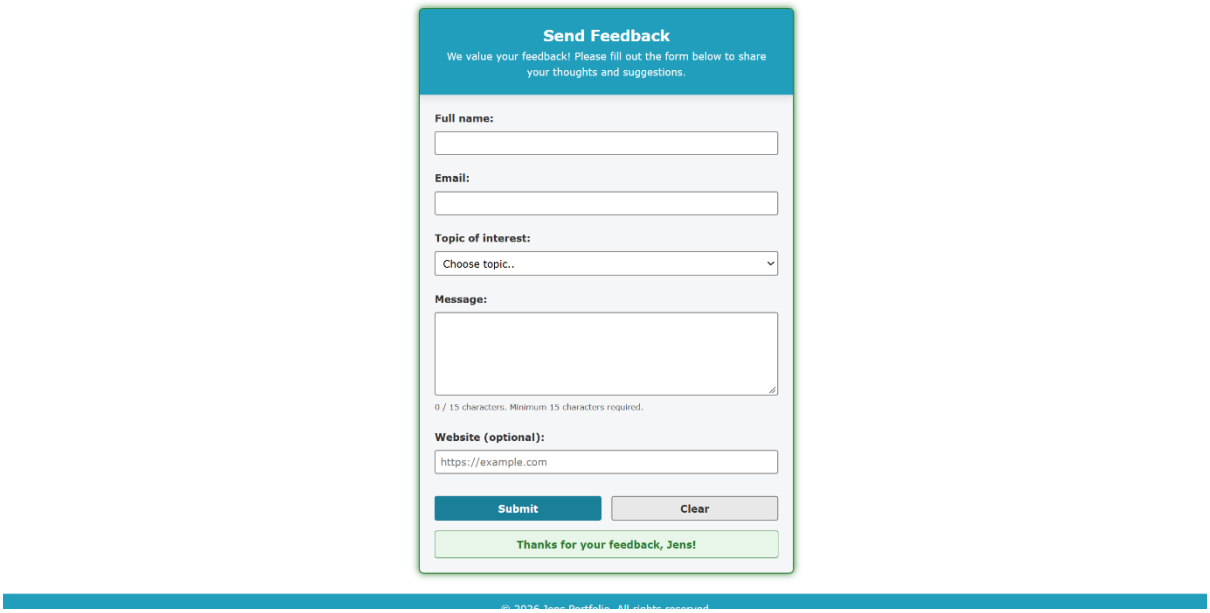


Valid state: Field valid inputs with name “Jens”, email jendo_23@hotmail.com, Topic selected “Bug Report”, Message “Test message over 15 chars” (27 characters), Website “https//:testwebsite.com”



© 2026 Jens Portfolio. All rights reserved.

Success state after valid submit including feedback message with users name



Code explanation

Form elements are selected once at the top with getElementById(). Stored in const. Code is split into functions, each with a clear task.

validateFullName(), validateEmail(), validateTopic(), validateMessage(), validateWebsite() = checks the field value and validates against specific rules. Return true or false.

showFieldError() = shows error message, adds invalid class putting it into the Red state.

clearFieldError() = removes error message, adds the valid class putting it into Green state.

resetFieldState() = removes error message and invalid/valid class, neutral state.

updateMessageCounter() = updates the text content and color class.

showSuccessMessage() = shows the “Thanks for your feedback, ‘Full Name’!” message, clearing after 4 seconds.

resetForm() = resets the whole form.

Validation is done both by Submit and blur giving the user immediate feedback on input.

On Submit, user clicks submit button and form submit event triggers. Event.preventDefault() so the page does not reload. Validators are triggered with result stored in variables such as “isNameValid”. Validators trigger showFieldError or clearFieldError function.

On blur validation is triggered when the user leaves a field and the event listener calls only that fields validator, for example validateFullName(). This gives the user immediate feedback on the input being valid or invalid.

Live character counter on the message input field works by every keystroke triggering the input event listener which calls updateMessageCounter(). The function counts the text and compares it with MIN_MESSAGE_LENGTH which is set to 15. The if, else if, else bracket chooses which state and class to use. Empty gives neutral state, under 15 is red state by adding class “not-ok” and 15+ is green state by adding class “ok”. The feedback to user is both by text and color.

Success message and reset when all validators are true. The full name must be stored before the form resets as it empties all fields.
form.reset() clears values and triggers reset event which triggers resetForm() -> resetFieldState() for each field. The showSuccessMessage() writes the message and takes in the stored full name, presenting it in the status area. setTimeout() clears it after 4 seconds.

Clear button is of type=”reset” which triggers reset event and resetForm(). Clearing all errors messages and states making the form neutral as the initial state.

Problem solving

Success message including Full name. First when I implemented this functionality the name showed as empty. The form was resetted meaning the input values were empty. I had to fetch and store the full name inside the function, before the form.reset() so I could use the value in the success message.

Immediate feedback to user on valid / invalid input. First when a user input was invalid, for example less than 2 characters on the full name, it gives the correct error message and state. But when the user adds characters making the full name field valid, meeting requirements the user did not get that feedback as the error message and state was till showing as invalid until Submit. By adding the event listener for blur on the fields that validation was done in real time after the user switches input fields giving user immediate feedback.

Reflection prompts

Which interaction are you most confident explaining, and why?

Live character counter is very straight forward. It uses one event (input), one function and one element is updated. Not dependent on validators or other logic.

How does your Git history show the way your solution changed during development?

I followed the instructions for Git commits in the assignment, keeping them clear and straight forward for what has been done. I have over 20 commits over several different

days. Starting with the setup and project structure. After that I tried to do commits in step by step for each functions such as field validation, with one field at a time, submit and clear functionality, visual states and so on. Changes can be seen from commits moving repeated textContent code to the functions showFieldError and clearFieldError functions. Commits showing the css styling of the page as well as html building. With some fixes to bugs found during testing.

If you had one more development session, what would you improve or test further?

The feedback form page has no navigation so adding that with links back to the homepage could be some improvent. But as the form opens in a new tab the user can just close the tab and be back at the homepage as well.

There is also added functionality in my code for screen readers such as “aria-describedby”, “aria-invalid”, role=”status” and so on but I have not used a screen reader for testing that. Testing the form as a keyboard only user could also be interesting.